منصة مكافحة الشائعات

حروب الجيل الرابع — مصر

Vercel | Railway | Gemini AI | MySQL | React | Node.js

توثيق المشروع الكامل

الإصدار 1.0 — إبريل 2026

الموقع: `https://frontend-nine-beige-13.vercel.app`

Backend: `https://backend-production-03bc.up.railway.app`

📋 فهرس المحتويات

1. نظرة عامة على المشروع وأهميته
2. هيكل المشروع والمعمارية
3. اللغات والتقنيات المستخدمة
4. صفحات الموقع ووظيفة أيقوناتها
5. الـ Backend — API Endpoints
6. قاعدة البيانات — جداول ومزامنة

7. الذكاء الاصطناعي — Gemini Integration
8. نظام المصادقة والصلاحيات
9. النشر والبنية التحتية
10. المتغيرات البيئية
11. هيكل الملفات الكامل
12. دليل المشرف

1. نظرة عامة على المشروع وأهميته

منصة مكافحة الشائعات هي تطبيق ويب متكامل تم تطويره لمواجهة ظاهرة الأخبار الزائفة والمعلومات المضللة في المجتمع المصري والعربي، خاصةً في إطار ما يُعرف بـ **"حروب الجيل الرابع"** — وهي الحروب الإعلامية والمعلوماتية التي تستهدف إضعاف المجتمعات من الداخل عبر نشر الذعر، التفرقة، وتشويه الحقائق.

> 💡 **أهمية المشروع:**
> تُشير الإحصاءات إلى أن 68% من مستخدمي الإنترنت في مصر يتعرضون لأخبار زائفة يومياً عبر وسائل التواصل الاجتماعي. هذه المنصة توفر أداة مجتمعية وتقنية لمواجهة هذه الظاهرة بالتحليل الذكي والتوعية.

الأهداف الرئيسية للمنصة:

- 🔍 **التحقق الآلي** — تحليل الأخبار والشائعات باستخدام Gemini AI لتحديد مصداقيتها فورياً
- 📢 **الإبلاغ المجتمعي** — تمكين المواطنين من الإبلاغ عن الشائعات التي يتعرضون لها
- 📰 **التوعية والتثقيف** — نشر مقالات ومحتوى توعوي حول كيفية التعرف على الأخبار الزائفة
- 🎬 **الفيديوهات التوعوية** — عرض مقاطع فيديو من YouTube Facebookg TikTokg لنشر الوعي
- 🤖 **المساعد الذكي** — محادثة مباشرة مع Gemini AI للاستشارة حول أي خبر أو معلومة
- ⚙️ **لوحة إدارة** — تحكم كامل للمشرفين في المحتوى والمستخدمين والبلاغات

👥	👮	⚙️	🤖
المستخدمون	**المراجعون**	**المشرفون**	**Gemini AI**

مواطنون يريدون التحقق من الأخبار

متخصصون يراجعون البلاغات

إدارة كاملة للمنصة

تحليل ذكي تلقائي

2. هيكل المشروع والمعمارية

المشروع يعتمد نمط **Full-Stack Web Application** مع فصل كامل بين الـ Frontend والـ Backend:

```
منصة-مكافحة-الشائعات/
├── /fake-news-backend          # الخادم الخلفي (Node.js)
│   ├── index.js                # نقطة الدخول الرئيسية + جميع API Routes
│   ├── database.js             # إنشاء جداول قاعدة البيانات + بيانات أولية
│   ├── connection.js           # اتصال MySQL pool
│   ├── package.json            # تبعيات Node.js
│   ├── Dockerfile              # للنشر على Railway
│   ├── railway.json            # إعدادات Railway
│   ├── env.                    # متغيرات البيئة المحلية
│   └── /uploads                # ملفات مرفوعة (أدلة بلاغات)
│
├── /frontend                   # الواجهة الأمامية (React + Vite)
│   ├── /src
│   │   ├── main.jsx            # نقطة الدخول React
│   │   ├── App.jsx             # التطبيق الرئيسي + التنقل + Hamburger Menu
│   │   ├── api.js              # Axios instance + endpoints
│   │   ├── auth.jsx            # AuthContext (تسجيل دخول/خروج)
│   │   ├── styles.css          # تصميم كامل للموقع (RTL + Dark Theme)
│   │   ├── Login.jsx           # صفحة الدخول
│   │   ├── Register.jsx        # صفحة التسجيل
│   │   ├── ReportPage.jsx      # صفحة الإبلاغ عن شائعة
│   │   ├── ArticlesData.jsx    # صفحة عرض المقالات
│   │   ├── VideoAwareness.jsx  # صفحة فيديوهات التوعية
│   │   ├── AdvancedAIDetection.jsx # صفحة التحقق بالذكاء الاصطناعي
│   │   ├── AIChat.jsx          # صفحة الدردشة مع Gemini AI
│   │   ├── AdminDashboard.jsx  # لوحة تحكم المشرف
│   │   ├── AdminArticles.jsx   # إدارة المقالات (CRUD كامل)
│   │   ├── contentSettings.js  # إعدادات المحتوى (localStorage)
│   │   ├── ErrorBoundary.jsx   # معالجة الأخطاء
│   │   ├── LoadingSpinner.jsx  # مؤشر التحميل
│   │   ├── Notifications.jsx   # نظام الإشعارات
│   ├── index.html              # HTML الأساسي
│   ├── vite.config.js          # إعدادات Vite
│   ├── package.json            # تبعيات React
│   └── vercel.json             # SPA rewrite لـ Vercel
│
├── vercel.json                 # إعدادات Vercel على مستوى المشروع
└── README.md
```

نمط المعمارية (Architecture Pattern):

SPA
Single Page Application بدون إعادة تحميل

REST API
اتصال Frontend/Backend عبر HTTP/JSON

JWT Auth
توكنات مشفرة للمصادقة

Cloud Deploy
Railway + Vercel لنشر مستمر

3. اللغات والتقنيات المستخدمة

الطبقة	التقنية	الإصدار	الغرض
Frontend	React	18.2	بناء واجهة المستخدم التفاعلية
	JavaScript (JSX)	ES2022	لغة البرمجة الأساسية للواجهة
	Vite	5.4	أداة البناء والتطوير السريع
	React Router DOM	6.20	التنقل بين الصفحات (SPA)
	Axios	1.6	طلبات HTTP لـ Backend
	Bootstrap RTL	5.3	تصميم متجاوب للعربية
	CSS3	—	تصميم مخصص (Dark Theme + RTL)
Backend	Node.js	+18	بيئة تشغيل JavaScript على الخادم
	Express.js	4.18	إطار عمل HTTP/API
	JavaScript (CJS)	ES2022	لغة البرمجة لـ Backend
	mysql2	x.3	اتصال بقاعدة البيانات MySQL
	bcryptjs	x.2	تشفير كلمات المرور

	jsonwebtoken	x.9	إنشاء والتحقق من JWT
	multer	x.1	رفع الملفات (أدلة البلاغات)
	cors	x.2	السماح بطلبات Cross-Origin
	dotenv	x.16	قراءة المتغيرات البيئية
Database	MySQL	8.0	قاعدة البيانات العلائقية الرئيسية
	Railway MySQL	—	استضافة قاعدة البيانات في السحابة
AI	Google Gemini 1.5 Flash	—	تحليل الأخبار والدردشة الذكية
DevOps	Railway	—	استضافة وتشغيل الـ Backend
	Vercel	—	استضافة وبناء الـ Frontend
	GitHub	—	مستودع الكود + CI/CD
	Docker	—	حاوية لتشغيل الـ Backend

4. صفحات الموقع ووظيفة أيقوناتها

شريط التنقل (Navigation Bar)

الشريط يظهر في أعلى كل صفحة ويتحول إلى **قائمة Hamburger** ☰ على الشاشات الصغيرة (موبايل).

الأيقونة	النص	المسار (Route)	الوصول	الوظيفة
📢	إبلاغ	`report/`	مسجل دخول	صفحة الإبلاغ عن شائعة جديدة
📰	مقالات	`articles/`	مسجل دخول	قراءة مقالات التوعية من قاعدة البيانات
🔍	تحقق AI	`ai-detect/`	مسجل دخول	تحليل نص بالذكاء الاصطناعي
🤖	دردشة AI	`ai-chat/`	مسجل دخول	محادثة تفاعلية مع Gemini

🎬	فيديوهات	`videos/`	مسجل دخول	عرض فيديوهات توعوية من يوتيوب وغيره
⚙️	لوحة التحكم	`admin/`	مشرف فقط	إحصائيات ومراجعة البلاغات
📝	المقالات	`admin-articles/`	مشرف فقط	إضافة وتعديل وحذف المقالات
🔐	دخول	`login/`	غير مسجل	نموذج تسجيل الدخول
📝	تسجيل	`register/`	غير مسجل	إنشاء حساب جديد
🚪	خروج	—	مسجل دخول	تسجيل الخروج ومسح التوكن

شرح كل صفحة بالتفصيل:

📢 صفحة الإبلاغ — `report/`

- نموذج يتضمن: العنوان، التفاصيل، رابط المصدر، التصنيف (صحة/سياسة/اقتصاد/أمن/دين/عام)، الوسوم
- يرسل البيانات إلى `POST /api/reports`
- يدعم رفع ملفات كأدلة (صور، PDF) — حتى 5 ملفات × 10MB
- يظهر رسالة نجاح خضراء بعد الإرسال

📰 صفحة المقالات — `articles/`

- تجلب المقالات من `GET /api/articles` في الوقت الفعلي
- بحث فوري في العنوان والمحتوى
- عرض بطاقات Grid تحتوي: الفئة، العنوان، مقتطف، المؤلف، التاريخ

🔍 صفحة التحقق `/ai-detect` — AI

- حقل نصي لإدخال الخبر أو الشائعة
- يرسل إلى `POST /api/verify-ai`
- يعرض: نسبة المصداقية (0-100%)، حكم (صحيح/مشبوه/شائعة)، ملخص، أسباب
- إذا توفر GEMINI_API_KEY: تحليل حقيقي بـ Google Gemini 1.5 Flash
- إذا لم يتوفر: تحليل محلي بناءً على كلمات الشك

🤖 صفحة الدردشة `/ai-chat` — AI

- واجهة دردشة تفاعلية مثل WhatsApp
- يرسل رسائل إلى `POST /api/ai-chat`
- Gemini يجيب بالعربية عن أي سؤال يخص الشائعات والمعلومات المضللة
- يحفظ سجل المحادثة خلال الجلسة الواحدة

🎬 صفحة الفيديوهات — `videos/`

- يجلب الفيديوهات من `GET /api/videos`
- YouTube: يعرض Player مدمج iframe مباشرة
- TikTok / Facebook / أخرى: زر للانتقال إلى الرابط
- المشرف فقط: يرى زر "+ إضافة فيديو" + زر "حذف" على كل فيديو

⚙️ لوحة التحكم — `admin/`

- 4 بطاقات إحصاء: إجمالي البلاغات، قيد المراجعة، المقالات، المصادر
- جدول أحدث 5 بلاغات مع حالتها
- يجلب من `GET /api/dashboard` و `GET /api/reports`

📝 إدارة المقالات — `admin-articles/`

- CRUD كامل: إضافة / تعديل / حذف المقالات
- حقول: العنوان، المحتوى، الفئة، الوسوم، حالة النشر
- جدول بجميع المقالات مع إجراءات لكل صف

5. الـ API Endpoints — Backend

Base URL: `https://backend-production-03bc.up.railway.app`

جميع الـ endpoints المحمية تتطلب `Authorization: Bearer <JWT_TOKEN>` :Header

🔓 عام (لا يحتاج توكن)

الطريقة	المسار	الوظيفة
`GET`	`/`	فحص تشغيل الـ API

GET	api/health/	فحص صحة الخادم + timestamp
POST	api/register/	تسجيل مستخدم جديد (name, email, password)
POST	api/login/	تسجيل الدخول — يُرجع JWT token
GET	api/articles/	جلب المقالات المنشورة

🔒 يحتاج توكن (مستخدم مسجل)

الطريقة	المسار	الوظيفة
GET	api/reports/	جلب البلاغات (فلترة: status, category, page, limit)
POST	api/reports/	إرسال بلاغ جديد + رفع ملفات أدلة
GET	api/reports/:id/	تفاصيل بلاغ واحد + ملفات الأدلة
PUT	api/reports/:id/	تحديث حالة البلاغ (pending/true/false)
POST	api/verify-ai/	تحليل نص بـ Gemini AI
POST	api/ai-chat/	محادثة مع Gemini AI
GET	api/videos/	جلب قائمة الفيديوهات التوعوية
GET	api/sources/	جلب المصادر الموثوقة

👑 يحتاج صلاحية مشرف (admin)

الطريقة	المسار	الوظيفة
DELETE	api/reports/:id/	حذف بلاغ
POST	api/articles/	إضافة مقال جديد

PUT	`api/articles/:id/`	تعديل مقال
DELETE	`api/articles/:id/`	حذف مقال
POST	`api/videos/`	إضافة فيديو توعوي
DELETE	`api/videos/:id/`	حذف فيديو
GET	`api/dashboard/`	إحصائيات لوحة التحكم
GET	`api/admin/users/`	قائمة جميع المستخدمين
PUT	`api/admin/users/:id/role/`	تغيير دور مستخدم

6. قاعدة البيانات — جداول ومزامنة البيانات

نوع قاعدة البيانات: **MySQL 8.0** مستضافة على **Railway**
اسم قاعدة البيانات: `railway`
Charset: `utf8mb4` (يدعم العربية والإيموجي)

الجداول وعلاقاتها:

1. جدول users — المستخدمون

العمود	النوع	الوصف
user_id (PK)	INT AUTO_INCREMENT	المعرف الفريد
name	VARCHAR(120)	الاسم الكامل
email	VARCHAR(160) UNIQUE	البريد الإلكتروني (فريد)
password_hash	VARCHAR(255)	كلمة المرور مشفرة بـ bcrypt (10 rounds)
role	ENUM	user / reviewer / admin / spokesperson

created_at	TIMESTAMP	تاريخ الإنشاء

2. جدول reports — البلاغات

العمود	النوع	الوصف
report_id (PK)	INT AUTO_INCREMENT	المعرف الفريد
user_id (FK)	INT → users	المستخدم الذي أرسل البلاغ
title	VARCHAR(255)	عنوان الشائعة
description	TEXT	تفاصيل الشائعة
source_url	VARCHAR(500)	رابط مصدر الشائعة
category	VARCHAR(120)	التصنيف (.../general/health/politics)
tags	VARCHAR(255)	وسوم للبحث
status	ENUM	pending / true / false
admin_notes	TEXT	ملاحظات المشرف بعد المراجعة
created_at / updated_at	TIMESTAMP	تواريخ الإنشاء والتحديث

3. جدول articles — المقالات

العمود	النوع	الوصف
article_id (PK)	INT AUTO_INCREMENT	المعرف الفريد
title	VARCHAR(255)	عنوان المقال
content	LONGTEXT	محتوى المقال الكامل
category	VARCHAR(120)	الفئة (.../awareness/education/health)

tags	VARCHAR(255)	وسوم البحث
is_published	TINYINT(1)	1=منشور، 0=مسودة
created_by (FK)	INT → users	المشرف الكاتب للمقال
created_at	TIMESTAMP	تاريخ النشر

4. جدول awareness_videos — فيديوهات التوعية

العمود	النوع	الوصف
video_id (PK)	INT AUTO_INCREMENT	المعرف الفريد
title	VARCHAR(255)	عنوان الفيديو
description	TEXT	وصف الفيديو
video_url	VARCHAR(500)	رابط الفيديو (YouTube/Facebook/TikTok)
platform	ENUM	youtube / facebook / tiktok / other
created_at	TIMESTAMP	تاريخ الإضافة

5. جدول report_evidence — أدلة البلاغات

يخزن الملفات المرفوعة (صور، PDF) كأدلة لكل بلاغ. علاقة: report_id → reports (CASCADE DELETE)

6. جدول sources — المصادر الموثوقة

قائمة المصادر الإعلامية الموثوقة مع درجة مصداقيتها (credibility_score من 0-100)

7. جدول verifications — التحقيقات

نتائج التحقق من البلاغات من قِبَل المراجعين (verdict: true/false)

مزامنة البيانات (Data Synchronization):

✅ **آلية المزامنة في المشروع:**

1. كل طلب Frontend يستخدم **Axios Instance** مع `baseURL` ثابت
2. الـ JWT token يُرسل تلقائياً مع كل طلب عبر **Request Interceptor**
3. قاعدة البيانات على Railway تعمل **24/7** مع connection pooling
4. صفحة المقالات والفيديوهات تجلب البيانات في **Real-Time** عند كل زيارة
5. لوحة التحكم تجلب الإحصائيات من الـ API مباشرةً (لا cache محلي للإحصائيات)
6. contentSettings.js تحفظ في **localStorage** وتُزامَن مع واجهة المشرف

7. الذكاء الاصطناعي — Gemini Integration

نموذج الذكاء الاصطناعي:

المنصة تستخدم **Google Gemini 1.5 Flash** عبر الـ REST API الرسمي من Google.

التحقق من الشائعات (`api/verify-ai/`):

يُرسل الـ Backend prompt محكم إلى Gemini يطلب منه تحليل النص وإرجاع JSON بـ:

- `confidence` : نسبة صحة المعلومة (0-100)
- `verdict` : صحيح / مشبوه / شائعة
- `summary` : ملخص التحليل بالعربية
- `reasons` : قائمة أسباب الحكم

Fallback بدون API Key:

إذا لم يُضف `GEMINI_API_KEY` في Railway، يعمل النظام بـ**تحليل محلي** يبحث عن كلمات مثل: "عاجل"، "خطير"، "انتشار"، "خبر مهم"، إلخ — لكن الدقة محدودة. يُنصح بإضافة المفتاح للحصول على تحليل حقيقي.

الحصول على GEMINI_API_KEY:

1. افتح `https://aistudio.google.com/app/apikey`
2. اضغط "Create API Key"
3. انسخ المفتاح

4. اذهب إلى Variables ← backend service ← Railway ← أضف: `GEMINI_API_KEY = مفتاحك`

8. نظام المصادقة والصلاحيات

تدفق تسجيل الدخول:

```
المستخدم يدخل email + password
↓
Frontend يرسل POST /api/login
↓
Backend يتحقق من البريد في DB
↓
bcrypt.compare(password, hash)
↓
jwt.sign({ userId, email, role, name }, SECRET, { expiresIn: '7d' })
↓
يُرجع: { token, role, userId, name }
↓
Frontend يحفظ في localStorage:
token | role | name | userId
↓
Axios interceptor يضيف تلقائياً:
<Authorization: Bearer <token
```

مستويات الصلاحيات:

الدور	الصلاحيات
user	تسجيل دخول، إرسال بلاغ، قراءة مقالات، التحقق AI، دردشة AI، مشاهدة فيديوهات
reviewer	كل صلاحيات user + مراجعة البلاغات وتحديث حالتها
admin	كل الصلاحيات + إدارة المقالات، الفيديوهات، المستخدمين، لوحة التحكم الكاملة
spokesperson	دور خاص للمتحدث الرسمي للمراجعة والتعليق

حماية الـ Routes:

- `isAuthenticated` Frontend: يتحقق من وجود token في localStorage
- `isAdmin` Frontend: يتحقق أن 'role === 'admin
- `auth` Backend: Middleware يتحقق من JWT في كل طلب محمي
- `admin` Backend: Middleware يتحقق أن 'role === 'admin

9. النشر والبنية التحتية

🚂
Railway
Backend Node.js + MySQL Database

▲
Vercel
Frontend React + Build تلقائي

🐙
GitHub
مستودع الكود + CI/CD

🐳
Docker
حاوية الـ Backend على Railway

روابط النشر:

الخدمة	الرابط
الموقع (Frontend)	`https://frontend-nine-beige-13.vercel.app`
الـ API (Backend)	`https://backend-production-03bc.up.railway.app`
فحص الـ API	`https://backend-production-03bc.up.railway.app/api/health`
GitHub	`https://github.com/faresosama200/rumor-detection-platform-egypt`

عملية النشر التلقائي:

1. `git push origin main` → GitHub يستقبل الكود
2. Vercel يكتشف التغيير تلقائياً → يبني الـ (`npm run build`) Frontend → ينشر
3. `railway up --detach` → يرفع الكود إلى Railway ويبني الـ Docker image → يشغّل

10. المتغيرات البيئية

Backend (Railway Variables):

المتغير	القيمة / الوصف	مطلوب؟
`MYSQLHOST`	عنوان خادم MySQL (يُضاف تلقائياً من Railway)	✅ تلقائي
`MYSQLPORT`	منفذ MySQL (افتراضي 3306)	✅ تلقائي
`MYSQLUSER`	اسم مستخدم MySQL	✅ تلقائي
`MYSQLPASSWORD`	كلمة مرور MySQL	✅ تلقائي
`MYSQLDATABASE`	اسم قاعدة البيانات (railway)	✅ تلقائي
`PORT`	5000 (أو يُحدد تلقائياً)	تلقائي
`JWT_SECRET`	مفتاح تشفير التوكنات (غيّره في الإنتاج!)	⚠️ مهم
`GEMINI_API_KEY`	مفتاح Google Gemini API	⚡ للذكاء الاصطناعي

Frontend (Vercel Environment Variables):

المتغير	القيمة	الوصف
`VITE_API_URL`	`https://backend-production-03bc.up.railway.app`	رابط الـ Backend (اختياري، الافتراضي هو نفس الرابط)

11. هيكل الملفات الكامل وما يفعله كل ملف

الملف	اللغة	الوظيفة
`fake-news-backend/index.js`	Node.js / JavaScript	الخادم الكامل: Express app + جميع الـ 20 API route + Gemini integration + Auth middleware

`fake-news-backend/database.js`	Node.js / JavaScript	إنشاء جداول قاعدة البيانات عند أول تشغيل + بيانات أولية (admin account + مقالات تجريبية)
`fake-news-backend/connection.js`	Node.js / JavaScript	MySQL connection pool للاستخدام المشترك
`fake-news-backend/package.json`	JSON	تبعيات Node.js: express, mysql2, bcryptjs, jsonwebtoken, multer, cors, dotenv
`fake-news-backend/Dockerfile`	Docker	حاوية تشغيل الـ Backend على Railway
`fake-news-backend/railway.json`	JSON	إعدادات النشر على Railway (startCommand)
`frontend/src/main.jsx`	React/JSX	نقطة دخول React — يلف التطبيق بـ AuthProvider
`frontend/src/App.jsx`	React/JSX	التطبيق الرئيسي: شريط التنقل + Hamburger Menu + جميع الـ Routes
`frontend/src/api.js`	JavaScript	Axios instance مع baseURL + Request interceptor يضيف JWT تلقائياً + قائمة endpoints
`frontend/src/auth.jsx`	React/JSX	AuthContext: حالة المستخدم + logout() + register() + login()
`frontend/src/styles.css`	CSS3	التصميم الكامل للموقع: Dark Theme، RTL، Responsive، Hamburger، Chat، AI Results
`frontend/src/Login.jsx`	React/JSX	صفحة تسجيل الدخول مع validation وعرض بيانات تجريبية
`frontend/src/Register.jsx`	React/JSX	صفحة إنشاء حساب جديد
`frontend/src/ReportPage.jsx`	React/JSX	نموذج الإبلاغ عن شائعة مع جميع الحقول
`frontend/src/ArticlesData.jsx`	React/JSX	عرض المقالات من الـ API مع بحث فوري وعرض Grid
`frontend/src/VideoAwareness.jsx`	React/JSX	عرض الفيديوهات مع YouTube embed + إضافة/حذف للمشرف
`frontend/src/AdvancedAIDetection.jsx`	React/JSX	واجهة التحقق AI: إدخال نص + عرض نتيجة Gemini مع نسبة ومؤشر بصري
`frontend/src/AIChat.jsx`	React/JSX	واجهة دردشة مع Gemini AI: فقاعات رسائل + scroll + typing indicator تلقائي
`frontend/src/AdminDashboard.jsx`	React/JSX	لوحة تحكم: 4 إحصائيات + جدول أحدث البلاغات
`frontend/src/AdminArticles.jsx`	React/JSX	CRUD كامل للمقالات: إضافة/تعديل/حذف مع جدول وفورم
`frontend/src/contentSettings.js`	JavaScript	Hook لإعدادات المحتوى (اسم المنصة، الفئات، الروابط) مع localStorage sync

`frontend/src/ErrorBoundary.jsx`	React/JSX	معالجة الأخطاء غير المتوقعة مع رسالة للمستخدم وزر إعادة تحميل
`frontend/src/LoadingSpinner.jsx`	React/JSX	مؤشر التحميل المتحرك مع رسالة مخصصة
`frontend/src/Notifications.jsx`	React/JSX	نظام إشعارات منبثقة (toast) مع Context
`frontend/vercel.json`	JSON	إعادة توجيه SPA: كل المسارات → index.html (منع خطأ 404)
`frontend/vite.config.js`	JavaScript	إعدادات Vite: React plugin + port 3000
`frontend/package.json`	JSON	تبعيات Frontend: react, react-dom, react-router-dom, axios, bootstrap, vite

12. دليل المشرف

🔑 **بيانات الدخول الافتراضية للمشرف:**

البريد: `admin@platform.com`

كلمة المرور: `admin123`

يُنصح بتغيير كلمة المرور بعد أول دخول.

ما يستطيع المشرف فعله:

1. **إضافة مقالات توعوية** — عبر /admin-articles → "+ مقال جديد"
2. **تعديل وحذف المقالات** — من نفس الصفحة
3. **إضافة فيديوهات** — عبر /videos → "+ إضافة فيديو" (يدعم YouTube/Facebook/TikTok)
4. **مراجعة البلاغات** — عبر /admin → جدول أحدث البلاغات
5. **مشاهدة الإحصائيات** — عبر /4 → admin بطاقات إحصائية
6. **تغيير أدوار المستخدمين** — عبر API: PUT /api/admin/users/:id/role

إضافة Gemini AI (خطوة واحدة):

1. اذهب إلى: `https://railway.com` → تسجيل دخول
2. اختر المشروع → خدمة الـ Backend → Variables
3. أضف: **GEMINI_API_KEY** = مفتاحك من `aistudio.google.com`

4. Railway يُعيد النشر تلقائياً خلال دقيقتين
5. الآن تحقق AI ودردشة AI يعملان بالذكاء الاصطناعي الحقيقي

منصة مكافحة الشائعات — حروب الجيل الرابع - مصر
تم إنشاء هذا التوثيق تلقائياً — إبريل 2026
للطباعة كـ PDF: افتح في المتصفح ← Save as PDF ← Ctrl+P